# Name : Bhavesh Punjabi
# Roll no : 40

# Experiment 2: To Perform Data Preprocessing Using Python

## Aim

To perform **data preprocessing using Python** by handling missing values, removing duplicate records, encoding categorical data, and scaling numerical features.

## Objectives

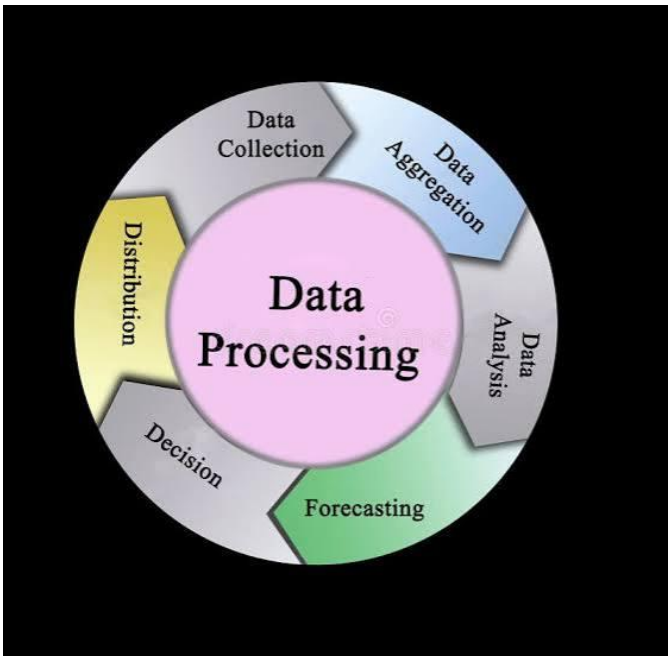
- Load a dataset using Python.
- Identify and handle missing values.
- Remove duplicate records.
- Convert categorical data into numerical form.
- Normalize/standardize numerical data.
- Prepare clean data for further analysis or machine learning.

## Software/Tools Required

- Python 3.x
- Jupyter Notebook / Google Colab
- Pandas
- NumPy
- Scikit-learn

## Theory

**Data Preprocessing** Data preprocessing is the process of converting raw data into a clean, consistent, and suitable format for analysis or machine learning. Real-world datasets may contain missing values, duplicate records, categorical attributes, and numerical features with different ranges. These issues can affect the accuracy and reliability of data analysis. Therefore, preprocessing is performed before applying analytical or machine-learning techniques.




Common preprocessing steps are:

1. **Data Cleaning** – Handling missing and incorrect values.
2. **Removing Duplicates** – Eliminating repeated records.
3. **Encoding** – Converting categorical values into numerical values.
4. **Feature Scaling** – Bringing numerical features to a similar scale.
5. **Data Transformation** – Converting data into a suitable format.

**Python Program :**

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, StandardScaler

# 1. Load Excel dataset
file_path = "/content/Movie_Ticket_Booking_Sample_Dataset.xlsx"
df = pd.read_excel(file_path)

print("===== ORIGINAL DATASET =====")
print(df)

# 2. Check missing values
print("\n===== MISSING VALUES =====")
print(df.isnull().sum())

# 3. Handle missing numerical value
```

```
df["Age"] = df["Age"].fillna(df["Age"].mean())

# 4. Handle missing categorical value
df["Payment_Method"] = df["Payment_Method"].fillna(
    df["Payment_Method"].mode()[0]
)

print("\n===== MISSING VALUES AFTER HANDLING =====")
print(df.isnull().sum())

# 5. Check duplicates
print("\n===== DUPLICATES =====")
print("Duplicate rows:", df.duplicated().sum())

# 6. Remove duplicates
df = df.drop_duplicates()

print("Duplicates after removal:", df.duplicated().sum())

# 7. Encode Gender
gender_encoder = LabelEncoder()
df["Gender"] = gender_encoder.fit_transform(df["Gender"])

print("\nGender Encoding:")
print(gender_encoder.classes_)

# 8. Encode Booking Status
status_encoder = LabelEncoder()
df["Booking_Status"] = status_encoder.fit_transform(
    df["Booking_Status"]
)

print("\nBooking Status Encoding:")
print(status_encoder.classes_)

# 9. Select numerical columns
numerical_columns = [
    "Age",
    "Tickets",
    "Ticket_Price",
    "Total_Amount"
]
```

```
# 10. Feature scaling
scaler = StandardScaler()

df[numerical_columns] = scaler.fit_transform(
    df[numerical_columns]
)

# 11. Display final dataset
print("\n===== FINAL PREPROCESSED DATASET =====")
print(df)

# 12. Final verification
print("\n===== FINAL CHECK =====")
print("Missing values:")
print(df.isnull().sum())

print("\nDuplicate rows:")
print(df.duplicated().sum())

# 13. Save preprocessed dataset
output_file = "/content/Preprocessed_Movie_Ticket_Booking.xlsx"

df.to_excel(output_file, index=False)

print("\nPreprocessed dataset saved successfully!")
```

**Expected Output :**

1.Loading data

```
Dataset Loaded Successfully!
  Booking_ID Customer_Name              Movie   Age  Gender    City  \
0       B001          Amit  Avengers: Endgame  22.0    Male  Mumbai
1       B002         Rahul          Inception  25.0    Male    Pune
2       B003         Priya             Dangal   NaN  Female  Mumbai
3       B004         Sneha           3 Idiots  24.0  Female  Nashik
4       B005        Vikram          Inception  30.0    Male    Pune

             Theatre Show_Time Ticket_Type  Tickets  Ticket_Price  \
0       PVR Phoenix  10:00 AM     Regular        2           180
1         INOX Pune  01:00 PM     Premium        3           300
2       PVR Andheri  06:30 PM     Regular        1           180
3  Cinepolis Nashik  09:00 PM         VIP        2           450
4         INOX Pune  03:30 PM     Premium        4           300

  Payment_Method Booking_Status  Total_Amount
0            UPI      Confirmed           360
1    Credit Card      Confirmed           900
2            UPI      Confirmed           180
3     Debit Card      Cancelled           900
4           Cash      Confirmed          1200
```

2. Dataset Size

```
print("Number of Rows:", df.shape
print("Number of Columns:", df.sh
```

```
Number of Rows: 21
```

3. Checking Duplicate and Missing values

```
print("Missing Values:")
print(df.isnull().sum())
```

```
Missing Values:
Booking_ID        0
Customer_Name     0
Movie             0
Age               2
Gender            0
City              0
Theatre           0
Show_Time         0
Ticket_Type       0
Tickets           0
Ticket_Price      0
Payment_Method    1
Booking_Status    0
Total_Amount      0
dtype: int64
```

```
print("Number of Duplicate Rows:", df.duplicated().sum())
```

```
Number of Duplicate Rows: 1
```

```
print(df[df.duplicated()])
```

```
   Booking_ID Customer_Name              Movie   Age Gender    City  \
20       B001          Amit  Avengers: Endgame  22.0   Male  Mumbai

        Theatre Show_Time Ticket_Type  Tickets  Ticket_Price Payment_Method  \
20  PVR Phoenix  10:00 AM     Regular        2           180            UPI

   Booking_Status  Total_Amount
20      Confirmed           360
```

```
df = df.drop_duplicates()
```

```
print("Duplicate Rows After Removal:", df.duplicated().sum())
```

```
Duplicate Rows After Removal: 0
```

4.Encoding categorial data

```
label_encoder = LabelEncoder()

df["Gender"] = label_encoder.fit_transform(df["Gender"])
print(df[["Customer_Name", "Gender"]].head(10))
```

```
  Customer_Name  Gender
0          Amit       1
1         Rahul       1
2         Priya       0
3         Sneha       0
4        Vikram       1
5          Neha       0
6         Arjun       1
7          Riya       0
8         Karan       1
9         Pooja       0
```

5.Feature scaling

```
numerical_columns = [
    "Age",
    "Tickets",
    "Ticket_Price",
    "Total_Amount"
]

print(df[numerical_columns].head())
```

```
         Age  Tickets  Ticket_Price  Total_Amount
0  22.000000        2           180           360
1  25.000000        3           300           900
2  25.947368        1           180           180
3  24.000000        2           450           900
4  30.000000        4           300          1200
```

6.Final Preprocessed Data

```
print("Final Preprocessed Dataset:")
print(df.head())
```

```
Final Preprocessed Dataset:
  Booking_ID Customer_Name              Movie       Age  Gender    City  \
0       B001          Amit  Avengers: Endgame -1.069122       1  Mumbai
1       B002         Rahul          Inception -0.295281       1    Pune
2       B003         Priya             Dangal -0.050911       0  Mumbai
3       B004         Sneha           3 Idiots -0.553228       0  Nashik
4       B005        Vikram          Inception  0.994453       1    Pune

            Theatre Show_Time Ticket_Type   Tickets  Ticket_Price  \
0       PVR Phoenix  10:00 AM     Regular -0.281718     -1.031242
1         INOX Pune  01:00 PM     Premium  0.845154      0.098886
2       PVR Andheri  06:30 PM     Regular -1.408590     -1.031242
3  Cinepolis Nashik  09:00 PM         VIP -0.281718      1.511547
4         INOX Pune  03:30 PM     Premium  1.972027      0.098886

  Payment_Method Booking_Status  Total_Amount
0            UPI      Confirmed     -0.795824
1    Credit Card      Confirmed      0.601721
2            UPI      Confirmed     -1.261672
3     Debit Card      Cancelled      0.601721
4           Cash      Confirmed      1.378134
```

**Conclusive Result — Business Strategy and Implementation**

Conclusion

The Movie Ticket Booking System can use the preprocessed booking data to support better business decisions. After handling missing values, removing duplicate bookings, encoding categorical information, and scaling numerical features, the data becomes more reliable and suitable for analysis and future machine-learning applications.

Business Strategy :

Based on the dataset, the business can adopt the following strategies:

1. Customer Segmentation
   Analyze customer age, gender, city, and booking behavior to identify different customer groups.

2. Movie Demand Analysis
   Compare the number of bookings for different movies to identify high-demand and low-demand movies.
3. Dynamic Pricing
   Ticket prices can be adjusted according to movie popularity, show timing, theatre, and demand.
4. Targeted Promotions
   Customer and booking information can be used to provide suitable discounts and offers for different customer segments.
5. Theatre and Show Optimization
   Booking trends can help management allocate more screens or popular show timings to high-demand movies.
6. Revenue Improvement
   Analysis of ticket quantity, ticket price, and total booking amount can help identify opportunities to increase revenue.

**Implementation**

The implementation can follow this process:

Movie Booking Data

↓

Data Collection

↓

Data Preprocessing

↓

Missing Value Handling

↓

Duplicate Removal

↓

Categorical Encoding

↓

Feature Scaling

↓

Data Analysis

↓

Business Decisions

↓

Improved Pricing, Promotions & Show Allocation

## Result

Thus, the Movie Ticket Booking System demonstrates how data preprocessing can convert raw booking information into clean and reliable data for business analysis. The processed data can help management understand customer behavior, movie demand, ticket sales, and revenue patterns, enabling better decisions regarding pricing, promotions, theatre allocation, and show scheduling. This provides a foundation for implementing data-driven business strategies and future machine-learning solutions.

==========

**GitHub Repository Link:**
https://github.com/Sentry2311/DMBI_EXPERIMENTS/tree/main/Experiment_02_Data_Preprocessing_Python

*(Contains complete documentation, dataset, and program code for Experiment 2: Data Preprocessing Using Python)*